DATABASE MANAGEMENT SYSTEMS

CHAPTER 7

Normalisation

A concise, exam-oriented guide to 1NF, 2NF and 3NF — advantages, disadvantages, rules, examples, and database anomalies.

TOPICS COVERED — SECTION 7.1

- Advantages & Disadvantages
- 1NF
- 2NF
- 3NF
- Rules with Examples
- Anomalies

DBMSCh. 7

Study Notes & Exam Companion
Beginner-friendly · Step-by-step · Concise

CHAPTER SEVEN

Normalisation

The systematic process of organising the columns and tables of a relational database to reduce data redundancy and improve data integrity.

7.1 Syllabus Scope

- Advantages and disadvantages of Normalisation
- First Normal Form (1NF)
- Second Normal Form (2NF)
- Third Normal Form (3NF)
- Rules of 1NF, 2NF and 3NF with examples
- Anomalies (Insertion, Update, Deletion)

Table of Contents

· Introduction to Normalisation 4

1 Advantages of Normalisation 6

2 Disadvantages of Normalisation 7

3 Anomalies (Insertion, Update, Deletion) 8

4 First Normal Form — 1NF (Rules & Example) 10

5 Second Normal Form — 2NF (Rules & Example) 11

6 Third Normal Form — 3NF (Rules & Example) 13

7 1NF → 2NF → 3NF Workflow 15

8 Comparison of 1NF, 2NF, 3NF 16

9 How to Identify the Normal Form 17

10 Common Mistakes 18

Introduction to Normalisation

Normalisation is a systematic technique of organising the data in a relational database so that redundancy (unnecessary repetition of data) is minimised and data integrity is maximised. It works by breaking a large, poorly structured table into two or more smaller, well-structured tables and defining relationships between them.

> ⭐ **Exam Point — Definition to Remember**
>
> Normalisation is the process of decomposing a relation into smaller relations that are free of undesirable anomalies, without losing any information.

Why is Normalisation Needed?

When data is stored carelessly in a single large table, the same fact often gets repeated many times (**data redundancy**). This wastes storage, and if the fact changes, every copy must be updated — if even one copy is missed, the database becomes inconsistent. This is the root cause of the *anomalies* covered in Section 3.

A Simple Real-World Example

Table 1.1 — A poorly designed, unnormalised table

StudentID	StudentName	CourseID	CourseName	Instructor
S1	Meera	C101	DBMS	Dr. Rao
S1	Meera	C102	Networks	Dr. Sen
S2	Arjun	C101	DBMS	Dr. Rao

The pair "S1 – Meera" is stored twice (once for each course she takes), and the pair "C101/DBMS – Dr. Rao" is also stored twice (once for each student taking it). If one copy were edited but not the other, the table would contradict itself. Normalisation fixes this by splitting the table into smaller tables — one for students, one for courses — so each fact is stored exactly once.



Figure 1.1 — Normalisation is a progressive, staged process. A table must satisfy each normal form before it is checked against the next one.

Table 1.1, Normalised

Splitting Table 1.1 into a `STUDENT` table, a `COURSE` table, and an `ENROLMENT` table removes every repeated fact — each student's name, and each course's name and instructor, is now stored exactly once:

STUDENT

StudentID	StudentName
S1	Meera
S2	Arjun

COURSE

CourseID	CourseName	Instructor
C101	DBMS	Dr. Rao
C102	Networks	Dr. Sen

ENROLMENT

StudentID	CourseID
S1	C101
S1	C102
S2	C101

"S1 – Meera" now appears once, in STUDENT, no matter how many courses she takes; and "C101/DBMS – Dr. Rao" now appears once, in COURSE, no matter how many students enrol in it. This is exactly what removes the redundancy and contradiction risk seen in Table 1.1.

Note: this ENROLMENT table has only the two key columns because Table 1.1 has no per-enrolment fact like marks; Section 5.2's ENROLMENT table also keeps a Marks column, since its source table includes one.

1 Advantages of Normalisation

- **Reduction of data redundancy** — each fact is stored only once (e.g. "DBMS – Dr. Rao" written a single time instead of once per student).
- **Reduction of duplicate data** — duplicate rows and repeated fact-combinations are naturally eliminated once related attributes are grouped together.
- **Improved data consistency** — a fact stored in one place can never disagree with itself.
- **Better data integrity** — primary/foreign keys enforce valid relationships between the smaller tables.
- **Easier maintenance** — smaller, focused tables are simpler to understand and modify.
- **Better organisation of data** — each table corresponds to a single real-world concept (Student, Course, Department, etc.).
- **Reduced possibility of update problems** — since a fact isn't repeated, it is far less likely that only some copies get updated.
- **Efficient modification of data** — e.g. correcting an instructor's office room means editing exactly one row instead of every row that mentions them.
- **Improved overall database design** — a cleaner, more logical schema that mirrors real-world entities.

> 💡 **Remember**
>
> Every advantage of Normalisation traces back to one root cause: **storing each fact exactly once**.

2 Disadvantages of Normalisation

Normalisation is highly beneficial, but decomposing a table into more and more smaller tables has practical costs:

- **Increased number of tables** — one unnormalised table may become several smaller tables.
- **More complex database structure** — more primary/foreign keys and relationships to track.
- **Need for more joins when retrieving data** — information that used to sit in one row is now spread across tables and must be joined back together.
- **Potentially more complex queries** — SQL queries can become longer and harder to write.
- **Performance considerations** — multiple joins add processing overhead, which can matter for very large tables or frequent queries.

> ⚠ **Common Mistake**
>
> Assuming "more Normalisation is always better." Normalisation keeps data clean and consistent, but excessive normalisation can make retrieval more complicated — the right level depends on how the data will be used.

Table 3.1 — Advantages vs Disadvantages of Normalisation

Advantages	Disadvantages
Reduces data redundancy	Increases the number of tables
Improves data consistency & integrity	Increases structural complexity
Easier maintenance of individual tables	Requires more joins to retrieve full information
Fewer insert/update/delete anomalies	Possible performance cost from multiple joins

3 Anomalies

A **database anomaly** is an inconsistency or error that arises when data in a poorly designed (unnormalised) table is inserted, updated, or deleted. Anomalies are a direct symptom of high data redundancy, and eliminating them is the main motivation for Normalisation.

Table 4.1 — EMP_PROJECT (unnormalised, used for all three anomalies below)

EmpID	EmpName	DeptName	DeptHead	ProjectID
E01	Nisha	Sales	Mr. Iyer	P10
E02	Rohit	Sales	Mr. Iyer	P11
E03	Kavya	HR	Ms. Bose	P12

3.1 Insertion Anomaly

Definition: A new fact cannot be inserted unless unrelated data is provided — data that may not even exist yet.

Why it occurs: DeptName/DeptHead are tied to an employee's row, so a department fact cannot exist independently.

> **Example**
>
> A brand-new department, "Finance," has no employees yet. There is no way to insert the row "Finance – Mr. Das" because every row requires an `EmpID` and `ProjectID`, which don't exist for this department.

3.2 Update Anomaly

Definition: A fact stored in multiple rows must be updated everywhere it appears — missing even one leaves the data inconsistent.

Why it occurs: `DeptHead` for "Sales" is repeated in both E01's and E02's rows.

> **Example**
>
> If Mr. Iyer is replaced by Ms. Nair as head of Sales, this must be changed in *every* row where DeptName = Sales. Missing E02's row leaves two different heads shown for the same department.

3.3 Deletion Anomaly

Definition: Deleting a row causes the unintended loss of other, unrelated information.

Why it occurs: Department facts exist only inside employee rows.

Example

If Kavya (E03) — the only HR employee — resigns and her row is deleted, the fact "HR is headed by Ms. Bose" is lost completely, even though it is logically unrelated to Kavya leaving.

How Normalisation helps in all three cases: separating department facts (DeptName, DeptHead) into their own table means they can be inserted independently, updated once, and are never lost when an employee row is deleted.

Table 4.2 — Comparison of the Three Anomalies

Anomaly	Meaning	Typical Problem
Insertion	Cannot add a new fact without adding unrelated/unavailable data	A new department cannot be added until an employee is hired for it
Update	A repeated fact must be changed in every occurrence	Forgetting one occurrence leaves contradictory data
Deletion	Deleting one fact unintentionally deletes another, unrelated fact	Losing department information when the last employee in it is removed

4 First Normal Form — 1NF

4.1 Definition & Rules

A relation is in **1NF** if every attribute contains only **atomic (single, indivisible) values**, with no repeating groups or multi-valued cells. Concretely:

- **Rule 1:** Each attribute must contain atomic/single values — no lists inside one cell.
- **Rule 2:** No repeating groups (e.g. columns like Phone1, Phone2, Phone3).
- **Rule 3:** Each field/cell holds exactly one value.
- **Rule 4:** Each row represents a single record.

4.2 Example — Before, Problem, Transformation, After

Table 5.1 — STUDENT_PHONE (violates 1NF)

Student	Phone Numbers
Amit	9876, 8765
Ravi	7654

Problem: Amit's cell holds two values — non-atomic, and hard to query (e.g. "find the student with phone 8765").

Transformation: give each phone number its own row.

Table 5.2 — STUDENT_PHONE, converted to 1NF

Student	Phone Number
Amit	9876
Amit	8765
Ravi	7654

⭐ **Exam Point**

1NF only deals with the *shape* of the data (how values are laid out in each cell) and atomicity (one indivisible value per cell) — not with which columns depend on which key. That concern begins at 2NF.

⚠ **Common Mistake**

Having *multiple rows* for the same student (Table 5.2) does **not** violate 1NF — 1NF only forbids multiple values *inside one cell*.

5 Second Normal Form — 2NF

5.1 Definition & Rules

A relation is in **2NF** if it is already in 1NF, and every non-key attribute depends on the **whole** primary key — not just part of it. 2NF only matters when the key is a **composite key** (two or more columns).

- **Rule 1:** The relation must already be in 1NF.
- **Rule 2:** No *partial dependency* — a non-key attribute depending on only part of a composite key.
- **Rule 3:** Non-key attributes must depend on the entire composite key.

> 📌 **Shortcut**
> A single-column key can never cause a 2NF violation — partial dependency requires a composite key.

5.2 Worked Example — STUDENT_COURSE

STUDENT_COURSE (StudentID, CourseID, StudentName, CourseName, Marks)

StudentID	CourseID	StudentName	CourseName	Marks
S1	C101	Meera	DBMS	88
S1	C102	Meera	Networks	91
S2	C101	Arjun	DBMS	76

Key:

`(StudentID, CourseID)` — composite, since a student takes many courses and a course has many students.

Partial dependencies:

`StudentName` depends only on StudentID; `CourseName` depends only on CourseID. `Marks` depends on both together, so it is fine. The table is **not in 2NF**.

Decomposition:

STUDENT	COURSE	ENROLMENT
(StudentID, StudentName)	(CourseID, CourseName)	(StudentID, CourseID, Marks)

Why this satisfies 2NF:

In each table, every non-key attribute now depends on the whole of its own key — StudentName on StudentID, CourseName on CourseID, and Marks on the full composite key (StudentID, CourseID).

6 Third Normal Form — 3NF

6.1 Definition & Rules

A relation is in **3NF** if it is already in 2NF, and no non-key attribute depends **transitively** on the key — i.e., every non-key attribute depends directly on the key, not through another non-key attribute. If Key → A and A → B, then B depends on the key only transitively (through A), which 3NF disallows. *(Read "Key → A" as "A depends on Key.")*

- **Rule 1:** The relation must already be in 2NF.
- **Rule 2:** Non-key attributes must not depend transitively on the key.
- **Rule 3:** Non-key attributes must depend *directly* on the key.

> 📌 **Shortcut**
>
> Ask: "Does one non-key column determine the value of another non-key column?" If yes, there is a transitive dependency and the table is not in 3NF.

6.2 Worked Example — EMPLOYEE

EMPLOYEE (EmployeeID, EmployeeName, DeptID, DeptName)

EmployeeID	EmployeeName	DeptID	DeptName
E01	Nisha	D1	Sales
E02	Rohit	D1	Sales
E03	Kavya	D2	HR

Key:

`EmployeeID` (single column — already in 1NF and 2NF, since no composite key exists).

Transitive dependency:

`EmployeeID → DeptID → DeptName`. So `DeptName` depends on EmployeeID only transitively, through the non-key attribute DeptID. The table is **not in 3NF**.

Decomposition:

EMPLOYEE	DEPARTMENT
(EmployeeID, EmployeeName, DeptID)	(DeptID, DeptName)

Why this satisfies 3NF:

In EMPLOYEE, EmployeeName and DeptID both depend directly on EmployeeID. In DEPARTMENT, DeptName depends directly on DeptID. No non-key attribute depends on another non-key attribute.

7 1NF → 2NF → 3NF Workflow



Figure 8.1 — Each stage assumes the table already satisfies the previous stage.

Table 8.1 — What Each Normal Form Requires and Fixes

Normal Form	Main Requirement	Main Problem Addressed
1NF	Atomic values; no repeating groups	Multiple/repeating values in a single cell
2NF	1NF + no partial dependency	Non-key attribute depending on only part of a composite key
3NF	2NF + no transitive dependency	Non-key attribute depending on another non-key attribute

8 Comparison of 1NF, 2NF and 3NF

Table 9.1 — 1NF vs 2NF vs 3NF (MCQ-Ready)

Feature	1NF	2NF	3NF
Basic requirement	Atomic values, no repeating groups	1NF + full dependency on the whole key	2NF + no dependency on non-key attributes
Depends on previous NF	None (base requirement)	Must satisfy 1NF first	Must satisfy 2NF first
Main issue addressed	Non-atomic / multi-valued cells	Partial dependency	Transitive dependency
Dependency concept	Not applicable	Non-key attribute → part of composite key	Non-key attribute → another non-key attribute
Typical violation	A cell with multiple values, e.g. "9876, 8765"	StudentName depending only on StudentID in a composite-key table	DeptName depending on DeptID, which depends on EmployeeID

9 How to Identify the Normal Form

Use this three-question method to examine any relation and determine its current normal form.

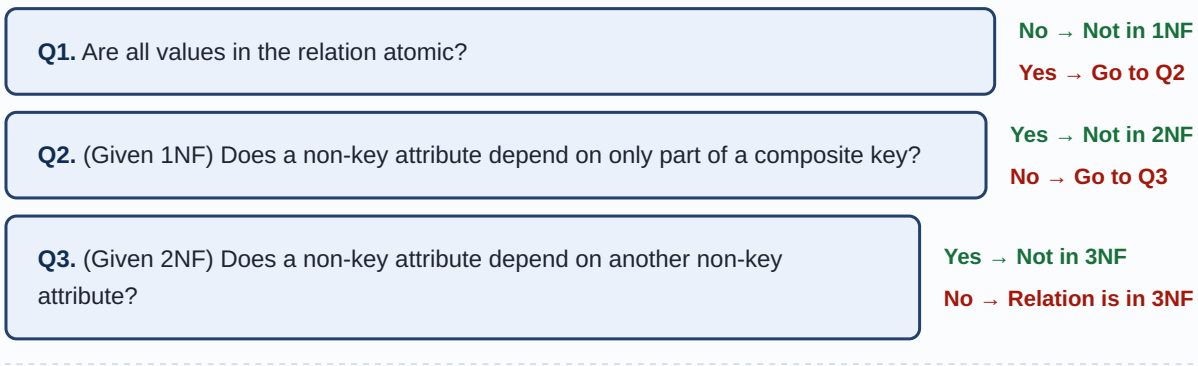


Figure 10.1 — Work through the questions strictly in order.

> 📌 **Shortcut**
>
> A single-column primary key automatically cannot violate 2NF — only Q1 and Q3 need checking in that case.

10 Common Mistakes

Table 11.1 — Frequent Mistakes and Corrections

Mistake	Correction
Thinking multiple rows for the same entity violate 1NF	1NF only concerns values *inside a cell*; multiple rows are expected.
Confusing "atomic" with "single-character"	Atomic means "not decomposed further" — "Meera" is atomic.
Applying 2NF/3NF checks out of order	Always confirm 1NF before 2NF, and 2NF before 3NF.
Confusing partial vs. transitive dependency	Partial = depends on *part of the key*. Transitive = depends on *another non-key attribute*.
Assuming every composite key violates 2NF	Only violates 2NF if some non-key column depends on *part* of the key, not the whole key.
Assuming every wide table violates 3NF	Only violates 3NF if a non-key column depends on another non-key column.